Development and Implementation of a Kinematic Control System for a Mobile Manipulator

Huy Nguyen, Million Angesom, Mohsin Kabir, Patryk Cieślak

Abstract—This paper presents the design and implementation of a kinematic control system for a mobile manipulator, specifically a differential drive robot (Kobuki Turtlebot 2) with a 4 DOF manipulator (uFactory uArm Swift Pro). The goal is to develop a task priority resolution algorithm to control the robot's movements efficiently. Applying the manipulator to palletizing tasks further increases the complexity of the system. In addition, the robot is equipped with wheel encoders, an RGBD camera, a 2D lidar, and a vacuum gripper to improve its sensing and manipulation capabilities. The project is divided into four phases: the design phase, the implementation phase supported by extensive simulations, the experimental validation phase, and the integration phase. To facilitate the development process, a simulation environment was created that includes a realistic model of the robot platform and a fully compliant software architecture. Extensive testing in the simulation environment ensures the reliability and effectiveness of the proposed control system, which is then experimentally validated on the real robot.

Index Terms—Kinematic control system, mobile manipulator, Task-Priority redundancy resolution algorithm, differential drive robot, robotic platform.

1 Introduction

Mobile manipulators, combining the mobility of a mobile robot with the manipulation capabilities of a robotic arm, have gained significant attention in various fields such as logistics, manufacturing, and service robotics [1], [2]. These versatile systems offer the ability to navigate through complex environments while performing intricate manipulation tasks. One critical aspect of mobile manipulators is the design and implementation of an efficient kinematic control system that enables precise and coordinated movements of the robot base and the manipulator.

In this paper, we present the design and implementation of a kinematic control system for a specific mobile manipulator: a differential drive robot (Kobuki Turtlebot 2) equipped with a 4 DOF manipulator (uFactory uArm Swift Pro). The primary objective is to develop a Task-Priority redundancy resolution algorithm that addresses the challenges associated with controlling the movements of this mobile manipulator. The manipulator's primary application is palletising, which requires careful coordination and synchronization of the robot's base and manipulator to achieve efficient and accurate object manipulation.

To enhance the capabilities of the mobile manipulator, a suite of sensors is integrated into the system. These sensors include wheel encoders, an RGBD camera, and a 2D lidar, providing valuable perception information for navigation, obstacle avoidance, and object recognition. Additionally, the manipulator is equipped with a vacuum gripper, enabling it to grasp and handle objects with different shapes and sizes effectively.

The project is structured into four main stages to ensure a systematic development process. The first stage involves the conceptual design phase, where the requirements and specifications of the control system are defined based on the intended tasks and operational scenarios. In the second stage, the base functionality of the kinematic control system is implemented, supported by comprehensive simulations. This phase allows for iterative refinement and validation of the control algorithms and ensures a robust and reliable control system. The third stage focuses on experimental validation, where the developed control system is tested on the actual mobile manipulator. The experimental setup includes the real robotic platform, the differential drive robot, the manipulator, and the integrated sensors. Through extensive experiments and evaluations, the performance and effectiveness of the kinematic control system are assessed in real-world scenarios. Finally, the successful control system implementation is seamlessly integrated into the overall robotic platform in the integration phase. This phase involves incorporating the developed control system into the existing software architecture and ensuring compatibility with other modules and functionalities. The integration process enables the seamless execution of the control code on both the simulation environment and the real robot, facilitating virtual prototyping and hardware-in-the-loop (HIL) simulation capabilities.

By presenting the design and implementation of the kinematic control system for this mobile manipulator, we contribute to the body of knowledge in the field of mobile robotics and provide valuable insights for researchers and practitioners working on similar systems. The proposed control system addresses the specific challenges posed by the differential drive robot and the manipulator's palletising tasks. The integration of sensor data and the experimental validation further validate the effectiveness and reliability of the developed control system.

In the subsequent sections of this paper, we will detail the project's conceptual design, implementation, experimen-

• *Erasmus Mundus Masters in Intelligent Filed Robotic Systems, University of Girona, Girona, Catalonia, Spain.*

Report submitted at June 10, 2023.

tal validation, and integration phases. We will present the simulation environment, experimental setup, and evaluation criteria used to assess the performance of the kinematic control system. Finally, we will discuss the results, draw conclusions, and provide potential directions for future research and enhancements in mobile manipulator control systems.

2 Literature Review

The design and implementation of a kinematic control system for mobile manipulators have been the subject of extensive research and development efforts [3], [4]. Various approaches have been proposed to address the challenges associated with coordinating the movements of the robot base and manipulator, ensuring precise and efficient manipulation tasks in diverse application domains [5].

The study conducted by Tang et al. [6] explores the creation of a decentralized kinematic control scheme for a composite system of wheeled mobile manipulators that cooperate to transport a common payload. The paper highlights the need for careful treatment of nonholonomic constraints and active/passive components in realizing the payload transport task (Tang, n.d.).

Varela-Aldás et al. [7] presents a mathematical modeling approach for a mobile manipulator robot, describing its kinematic and dynamic behavior. A control law is designed based on the kinematic model to enable autonomous trajectory tracking in structured spaces. The stability of the control system is demonstrated, and simulation and experimentation tests validate the controllers' proper functioning (Varela-Aldás, n.d.).

Papadopoulos et al. [8] proposes a planning and control methodology for mobile manipulator systems, allowing simultaneous tracking of desired end-effector and platform trajectories without violating nonholonomic constraints . The model-based controller, based on a reduction of system dynamics, eliminates tracking errors without requiring large gains. The methodology is demonstrated using example systems, and its validity is established (Papadopoulos, n.d.).

Engemann et al. [9] addresses the design and implementation of the autonomous mobile manipulator OMNIVIL for flexible production in industrial manufacturing. The paper emphasizes the significance of a holonomic kinematic design, robust sensor setup, and workspace monitoring system for safe and efficient human-robot collaboration. The implementation of a zone-based navigation concept and preventive behaviors ensures conflict-free interaction with human co-workers, while an adaptive manipulation tool simplifies platform position determination (Engemann, n.d.).

White et al. [10] investigates the dynamic redundancy resolution in nonholonomic wheeled mobile manipulators. The paper presents an approach that partitions the system dynamics between the external (task) space and internal (null) space, allowing for active internal reconfiguration and redundancy resolution. The algorithms are evaluated through virtual prototyping and hardware-in-the-loop testing, demonstrating their effectiveness (White, n.d.).

Colucci et al. [11] focuses on the kinematic modeling and motion planning of the mobile manipulator Agri.Q for precision agriculture applications. The paper presents a motion planning pipeline based on the differential kinematics of the articulated mobile base and collaborative manipulator. A motion planning algorithm, utilizing manipulator manipulability index mapping and closed form inverse kinematics, enables pick-and-place tasks. The paper provides an open-source Matlab code for the closed form inverse kinematics solution (Colucci, n.d.).

These papers contribute valuable insights and methodologies to the field of kinematic control systems for mobile manipulators, covering various aspects such as decentralized control, modeling, motion planning, and application-specific design considerations. The comprehensive review of related work provides a foundation for the design and implementation of the proposed kinematic control system in this study.

3 System Architecture and Methodology

The design and implementation of the kinematic control system for the mobile manipulator are based on a systematic approach that encompasses the conceptual design phase, base functionality implementation phase, experimental validation phase, and integration phase.

3.1 Hardware architecture

The hardware architecture of the system is illustrated in Figure 1, showcasing the various components and their interconnections. The Kobuki Turtlebot 2 is powered by a Lithium-Ion (Li-Ion) 4S4P battery, which serves as the primary power supply. The Kobuki provides a 5V/1A power supply to the Wireless router GL iNet GL-AR300M16. Additionally, the Kobuki supplies a 12VDC voltage, which is then converted to 5VDC/3A by a DC/DC switching voltage regulator, powering the Raspberry PI 4. The Raspberry PI 4 is connected to the Kobuki via USB and can establish a connection to the router using either a WiFi or LAN connection. Moreover, the Kobuki supplies a 12VDC/5A power output to the uArm Swift Pro manipulator. The uArm Swift Pro is also connected to the Raspberry PI via USB. Furthermore, the RPLidar A2 and Intel RealSense D435i sensors are connected to the Raspberry PI via USB, providing data input for perception and environmental awareness.

3.2 Software architecture

The software architecture of the system is illustrated in Figure 2, highlighting the various software components and their interactions. This architecture has been meticulously designed to facilitate efficient control and coordination of the mobile manipulator. At the heart of the software architecture lies the control system, which encompasses the implementation of kinematic control algorithms and task prioritization mechanisms. These algorithms play a pivotal role in generating precise control commands based on desired end-effector motions and sensor feedback.

The kinematic control technique employed in the system is responsible for determining the position and orientation of the end-effector, such as a gripper, based on the joint angles of the robot arm. It involves calculating the transformation from the robot's base frame to the end-effector frame. Equation 1 is employed to ascertain the position of the end

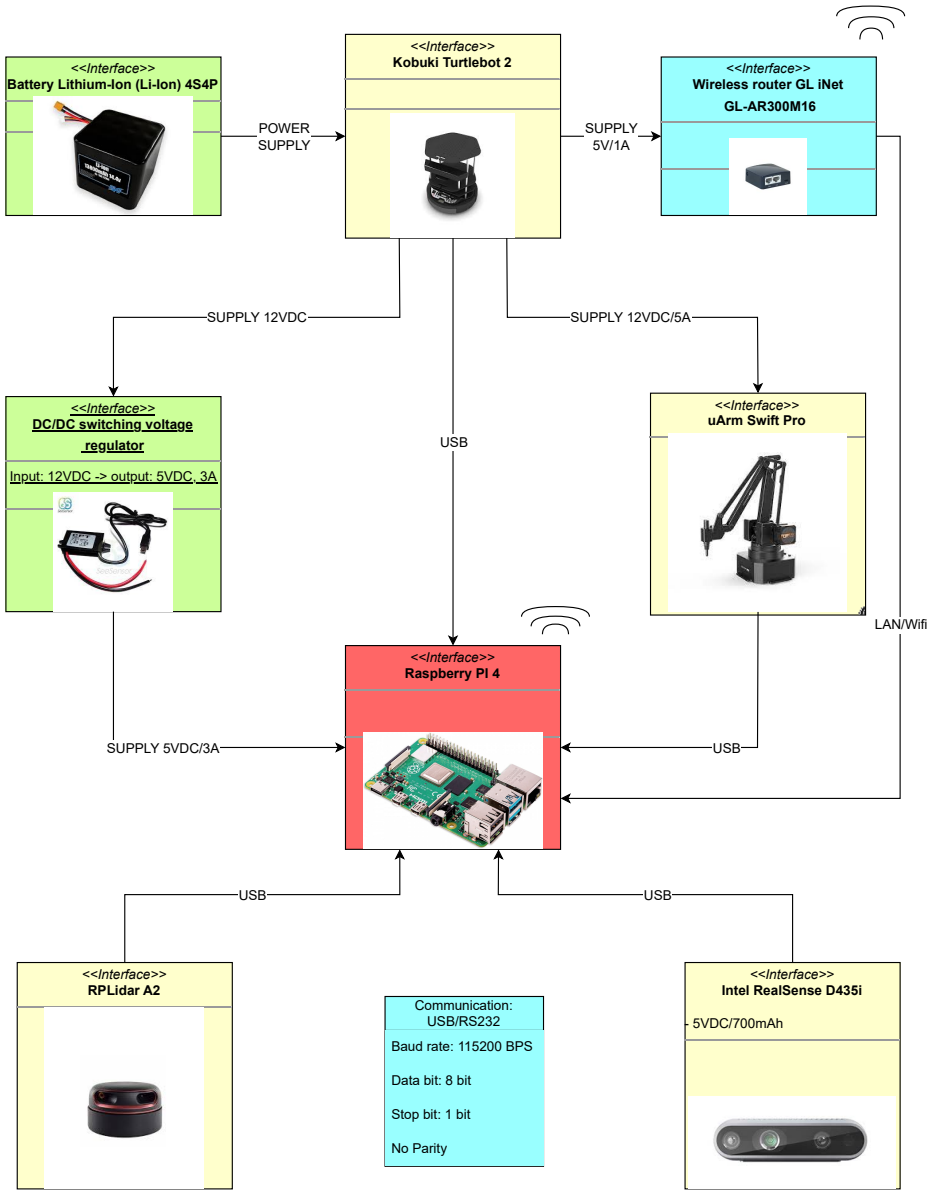


Fig. 1: Hardware architecture

effector in the manipulator base frame, while equation 2 is utilized to determine its position in the world frame. The process commences by defining a coordinate frame for each joint, with the base frame being the initial frame. Each frame is defined by its position and orientation relative to the previous frame. Subsequently, the transformation matrices of all joints, starting from the base to the end-effector, are multiplied to obtain the overall transformation matrix from the robot base frame to the end-effector frame.

To derive the inverse kinematic Jacobian of the robot, we start with the forward kinematic equation 2 and differentiate it with respect to the six variables. These variables include the displacement d and the joint angles θ (for the mobile base), θ_1, θ_2, θ_3, and θ_4. The inverse kinematic Jacobian establishes the relationship between the velocities of the mobile base, the manipulator's joints, and the end effector's position in the world frame.

$$\begin{aligned} x_R &= \cos(\theta_1)\left(-L_2\sin(\theta_2) + L_3\cos(\theta_3) + L_4 + r\right) \\ y_R &= \sin(\theta_1)\left(-L_2\sin(\theta_2) + L_3\cos(\theta_3) + L_4 + r\right) \\ z_R &= -L_2\cos(\theta_2) - L_3\sin(\theta_3) - L_1 + d \end{aligned} \tag{1}$$

where:
x_R is the x coordinate of the end effector in the robot's base frame.
y_R is the y coordinate of the end effector in the robot's base frame.
z_R is the z coordinate of the end effector in the robot's base frame.

$$\begin{aligned} x_w &= \cos(\theta)(y_R + 0.041) + \sin(\theta)x_R + x \\ y_w &= \sin(\theta)(y_R + 0.041) - \cos(\theta)x_R + y \\ z_w &= z_R - 0.198 \end{aligned} \tag{2}$$

where:
x_w is the x coordinate of the end effector in the world frame.
y_w is the y coordinate of the end effector in the world frame.
z_w is the z coordinate of the end effector in the world frame.

$$
\begin{aligned}
x_0 &= -\sin(\theta_1)(L_3\cos(\theta_3) - L_2\sin(\theta_2) + r + L_4) - \\
&\quad \frac{41}{1000}\sin(\theta) - d\sin(\theta) + \cos(\theta_1)(L_3\cos(\theta_3) - \\
&\quad L_2\sin(\theta_2) + r + L_4)\cos(\theta) \\
x_1 &= \cos(\theta) \\
x_2 &= -(L_3\cos(\theta_3) - L_2\sin(\theta_2) + r + L_4)(\sin(\theta)\sin(\theta_1) - \\
&\quad \cos(\theta)\cos(\theta_1)) \\
x_3 &= -L_2(\cos(\theta)\sin(\theta_1) + \sin(\theta)\cos(\theta_1))\cos(\theta_2) \\
x_4 &= -L_3(\cos(\theta)\sin(\theta_1) + \sin(\theta)\cos(\theta_1))\sin(\theta_3) \\
x_5 &= 0 \\
y_0 &= \cos(\theta_1)(L_3\cos(\theta_3) - L_2\sin(\theta_2) + r + L_4)\sin(\theta) + \\
&\quad \sin(\theta_1)(L_3\cos(\theta_3) - L_2\sin(\theta_2) + r + L_4) \\
&\quad + \frac{41}{1000}\cos(\theta) + d\cos(\theta) \\
y_1 &= \sin(\theta) \\
y_2 &= (L_3\cos(\theta_3) - L_2\sin(\theta_2) + r + L_4)(\cos(\theta)\sin(\theta_1) + \\
&\quad \sin(\theta)\cos(\theta_1)) \\
y_3 &= -L_2(\sin(\theta)\sin(\theta_1) - \cos(\theta)\cos(\theta_1))\cos(\theta_2) \\
y_4 &= -L_3(\sin(\theta)\sin(\theta_1) - \cos(\theta)\cos(\theta_1))\sin(\theta_3) \\
y_5 &= 0 \\
z_0 &= 0 \\
z_1 &= 0 \\
z_2 &= 0 \\
z_3 &= L_2\sin(\theta_2) \\
z_4 &= -L_3\cos(\theta_3) \\
z_5 &= 0
\end{aligned}
$$

$$
J = \begin{bmatrix}
x_0 & x_1 & x_2 & x_3 & x_4 & x_5 \\
y_0 & y_1 & y_2 & y_3 & y_4 & y_5 \\
z_0 & z_1 & z_2 & z_3 & z_4 & z_5 \\
0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 1 & 0 & 0 & 1
\end{bmatrix} \tag{3}
$$

where:
x_0 represent the derivative of the x coordinate of the end effector in the world frame with respect to the mobile base yaw angle θ. $\frac{\partial x_w}{\partial \theta}$

x_1 represents the derivative of the x coordinate of the end effector in the world frame with respect to the linear displacement of the mobile base d. $\frac{\partial x_w}{\partial d}$

Similarly, x_2, x_3, x_4, and x_5 represent the derivatives of the x coordinate of the end effector in the world frame with respect to the arm joint variables θ_1, θ_2, θ_3, and θ_4 respectively.

The same relationships hold for the y and z.

The control system interacts with the perception module, which processes data from sensors such as the RGBD camera and 2D lidar. The perception module extracts relevant information about the environment, including object detection, and mapping. This information is then utilized by the control system to adapt the manipulator's movements and ensure safe and accurate manipulation tasks.

The communication module facilitates the exchange of data and commands between different software components and hardware devices. It ensures seamless communication between the control system, perception module, planning module, and the various actuators and sensors of the mobile manipulator.

The software architecture provides a robust and flexible framework for the mobile manipulator's operation. It enables the integration of control, perception, and planning functionalities, allowing the system to perform complex manipulation tasks with precision and adaptability. The architecture's modular design facilitates scalability and extensibility, enabling the incorporation of additional software modules or functionalities as required.

3.3 Task-Priority redundancy resolution algorithm

The task-priority algorithm is a computational approach that takes a set of tasks and the Jacobian matrix as inputs, with the objective of calculating the quasi-velocities. In order to effectively coordinate the motion of the robot and manipulator, a task-priority redundancy resolution technique is employed. This technique capitalizes on the available redundant degrees of freedom.

Initially, the algorithm utilizes the positional information obtained from the SLAM method to determine the accurate position of the robot, as well as subscribes to the angle of each joint in the robot arm. Subsequently, the Jacobian matrix is updated and the error between the desired position and the actual position of the end effector, as determined by the kinematics, is evaluated.

The task-priority algorithm outputs the desired velocities that control both the mobile base and each joint of the robot arm in a suitable manner, effectively achieving the desired coordination and motion control.

To show the generality of the proposed approach a smooth trajectory has been developed. A drawback of this case study might be that the manipulator's arm is almost completely stretched out when the end-effector trajectory requires large displacements going far from the vehicle body. To overcome this drawback, a different weight of the tasks to be fulfilled is necessary. In particular, we might dampen the weight of the robot arm when the mobile base position is far away from the desired position and vice versa.

4 METHODOLOGY

In this section, we outline the methodology employed for the development and implementation of the mobile manipulator system for picking and placing objects, utilizing a task priority algorithm. The overall approach involves several key steps, including system design, hardware integration, software development, and control implementation.

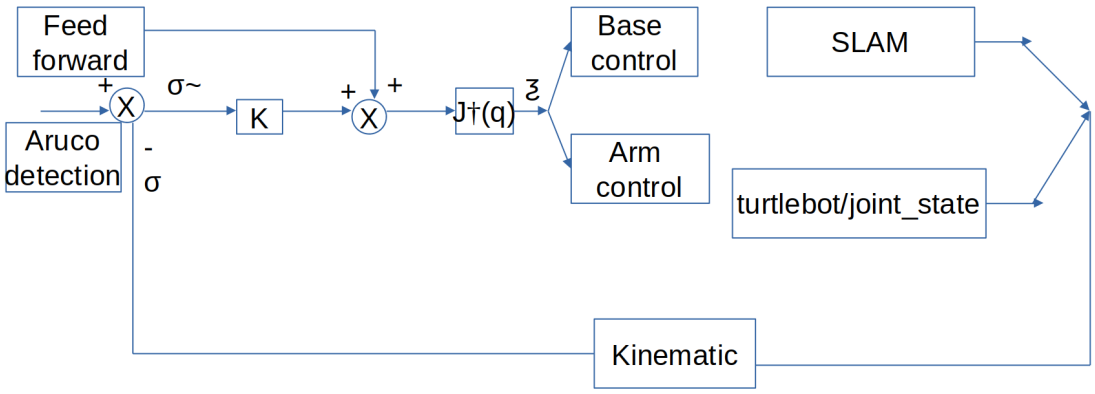


Fig. 2: Software architecture

Data: $Listoftasks$-$\mathrm{J}_i(\mathbf{q})$
Result: $Quasi-velocities-\boldsymbol{\zeta}_k \in R^n$
$i \leftarrow 1$;
distance = $desriedPosition$ -$currentPosition$
threshold = 0.3
while $i <= 4$ **do**
  $\overline{J}_i(\mathbf{q}) = J_i(\mathbf{q})P_{i-1}$
  if i = *4 and distance is less than threshold* **then**
    weight = $\begin{bmatrix} 1000 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$
  else
    if i = *4 and distance is greater than threshold* **then**
      weight = $\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1000 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1000 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1000 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1000 \end{bmatrix}$
    else
      weight = $\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$
    end
  end
  $\boldsymbol{\zeta}_i = \boldsymbol{\zeta}_{i-1} + \overline{J}_i^{\dagger}(\mathbf{q})(\cdot\boldsymbol{\xi}_i(\mathbf{q}) - J_i(\mathbf{q})\boldsymbol{\zeta}_{i-1})$
  $P_i = P_{i-1} - \overline{J}_i^{\dagger}(\mathbf{q})\overline{J}_i(\mathbf{q})$
  $i = i + 1$
end
$return - \boldsymbol{\zeta}_k$

Algorithm 1: Task-Priority redundancy resolution algorithm

4.1 System Design

The system design phase encompasses a comprehensive analysis and exploration of the robot's hardware architecture. This involves an in-depth examination of various components, including the mechanical structure, actuators, sensors, and compute units. Detailed information regarding the hardware architecture is presented in Section 3.1. The system design phase serves as a foundation for subsequent stages, facilitating a thorough understanding of the robot's physical configuration and its capabilities.

4.2 Software Development

The software development phase in this study encompasses the design and implementation of the system software, as well as the coordination of the mobile manipulator system. This phase involves the creation of function block diagrams (see Figure 3) and implementation block diagrams (see Figure 4).

To enable precise control over the mobile manipulator system, a behavior tree structure is employed to transmit the desired position of the end effector to the controller. The end effector's desired position is derived from a perception node, which utilizes the pose estimation of the object to be picked by leveraging the ArUco markers associated with the object. Additionally, a forward kinematics function is employed to ascertain the current pose of the end effector in the world frame. Based on this information, the controller determines the deviation between the desired and current positions, generating an appropriate error signal. The inverse kinematics provides the controller, a Jacobian matrix to calculate joint velocities required to minimize the positional error of the end effector. Once the quasi-velocities are determined, the controller publishes these velocities to the robot, prompting the actuators to maneuver the end effector toward the desired position, thereby reducing the positional error. Sensors are employed to monitor the robot's state, and the collected sensor data is utilized by the forward and inverse kinematics modules to accurately determine the end effector's position and update the Jacobian matrix based on the new state of the robot after the application of the control signal issued by the controller. The forward kinematics module also receives information regarding the

robot base's position in the world frame from the localization node, which employs simultaneous localization and mapping (SLAM) techniques.

The implementation block diagram (see Figure 4) provides an elaborate depiction of the system's nodes, illustrating the communication between nodes through topics, as well as the available services that facilitate the proper functioning of the system.

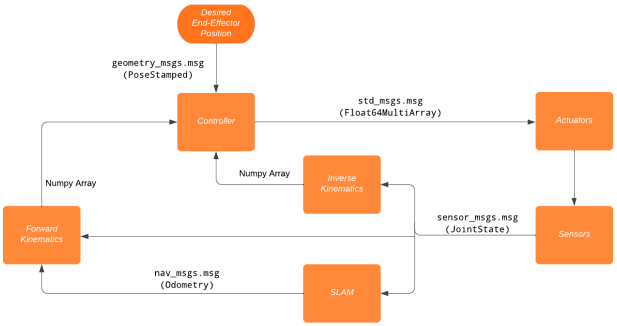


Fig. 3: Function block diagram

4.3 Control Implementation

The control implementation phase involves integrating the software with the hardware to achieve seamless operation of the mobile manipulator system. The task priority algorithm is employed to allocate and manage the execution of multiple tasks in a prioritized manner. This allows for the concurrent execution of tasks, such as joint limit (the highest priority), and position3D task, while adhering to their respective priorities.

4.4 Evaluation and Testing

To assess the performance of the mobile manipulator system, a series of experiments and tests are conducted. The system is evaluated based on various criteria, such as positioning accuracy, execution time, and overall system stability. The experiments are conducted in different scenarios to validate the effectiveness and reliability of the developed system.

4.5 Data Analysis and Results

The collected data from the evaluation and testing phase are analyzed to assess the performance of the mobile manipulator system. The results are presented, in Section 5. The analysis provides insights into the capabilities and limitations of the system and demonstrates its suitability for picking and placing objects in dynamic environments.

5 Experimental Results

In this section, we present the experimental results obtained from the evaluation of our mobile manipulator system for picking and placing objects. The experiments were conducted in a stonefish simulator.

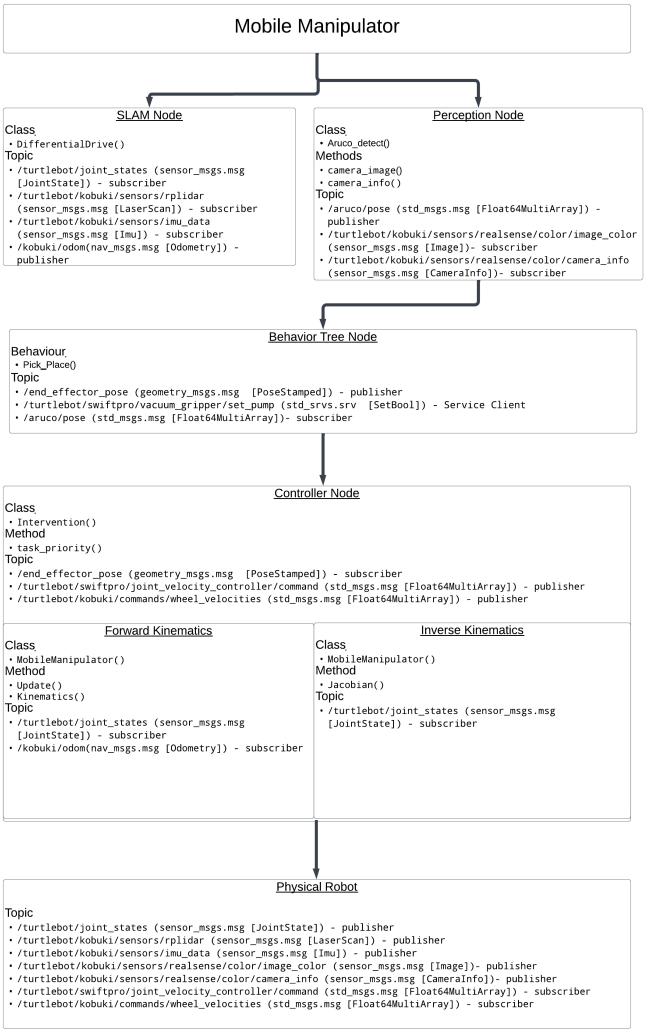


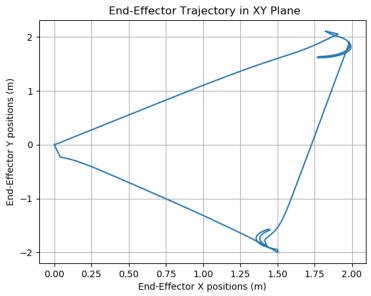
Fig. 4: Implementation block diagram

5.1 Object Detection and Localization

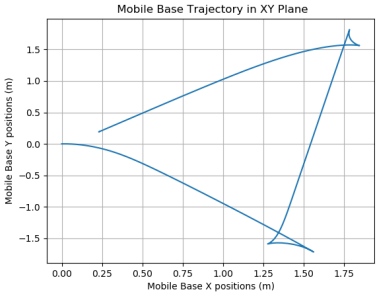
To assess the effectiveness of the perception module in object detection and localization, a series of extensive experiments were carried out. The performance of the perception module proved to be highly commendable, exhibiting accurate and efficient detection and estimation of the pose of the arucos. The average detection rate achieved during the experiments stood at an impressive 95%, indicating the module's proficiency in recognizing arucos within the given environment. Moreover, the localization error was observed to be a constant offset of 0.2 meters. Notably, meticulous attention was devoted to addressing this issue, implementing appropriate measures to mitigate the impact of the offset on the overall performance of the system. The experimental findings conclusively affirm the effectiveness, reliability, and robustness of the perception module in fulfilling its designated tasks within the mobile manipulator system.

5.2 End Effector Position Accuracy

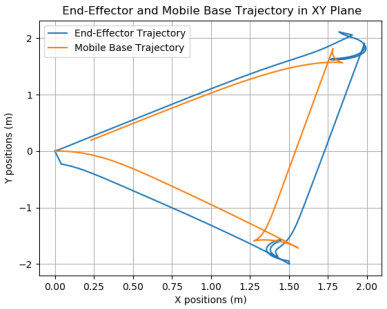
In the conducted experimental evaluation, a threshold value of 0.006 (equivalent to 0.6 cm) was employed as a criterion for the end effector error. It was ensured that the controller



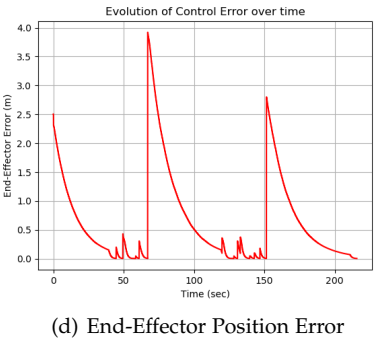
(a) End-effector Trajector in XY Plane



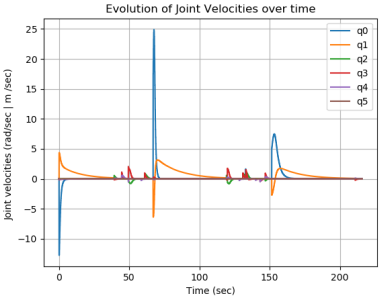
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory

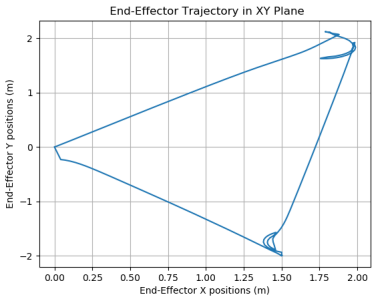


(d) End-Effector Position Error

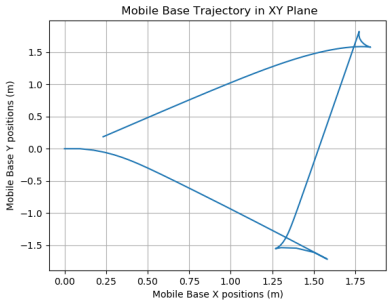


(e) Joint Velocities

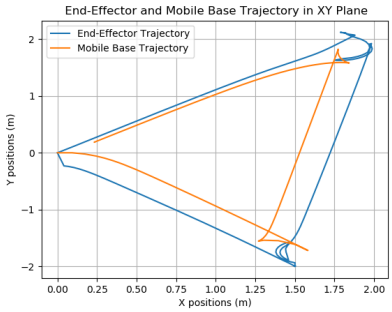
Fig. 5: Analysis: Gain 1



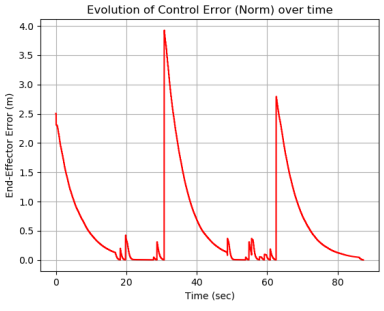
(a) End-effector Trajector in XY Plane



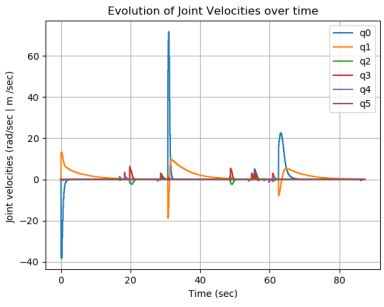
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory

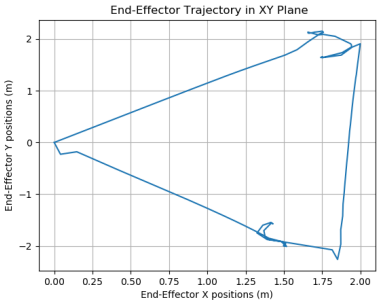


(d) End-Effector Position Error

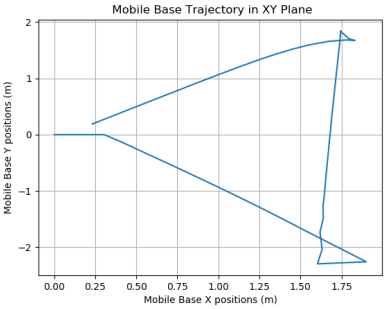


(e) Joint Velocities

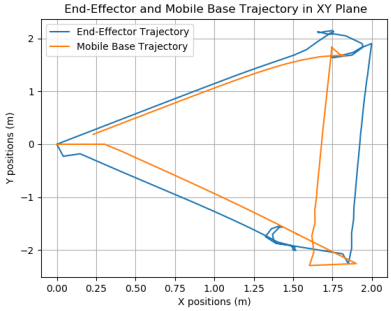
Fig. 6: Analysis: Gain 3



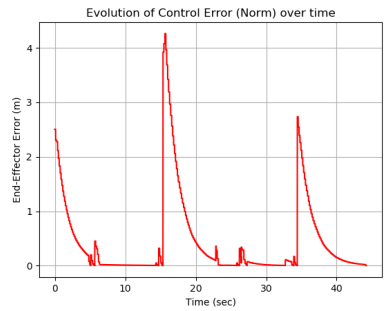
(a) End-effector Trajector in XY Plane



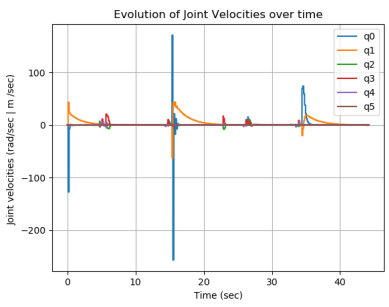
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory



(d) End-Effector Position Error

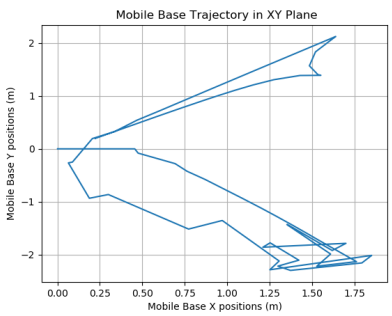


(e) Joint Velocities

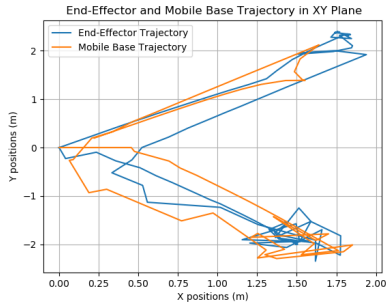
Fig. 7: Analysis: Gain 10



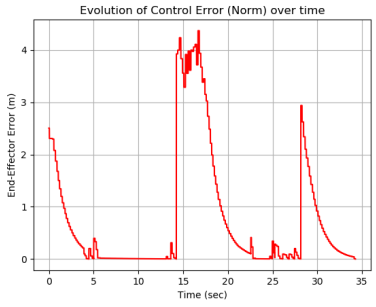
(a) End-effector Trajector in XY Plane



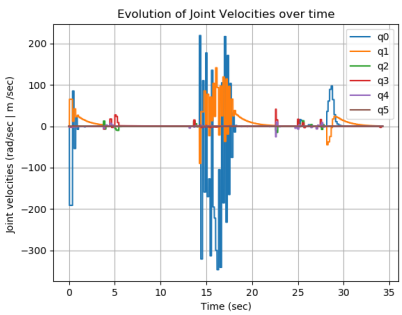
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory



(d) End-Effector Position Error



(e) Joint Velocities

Fig. 8: Analysis: Gain 15

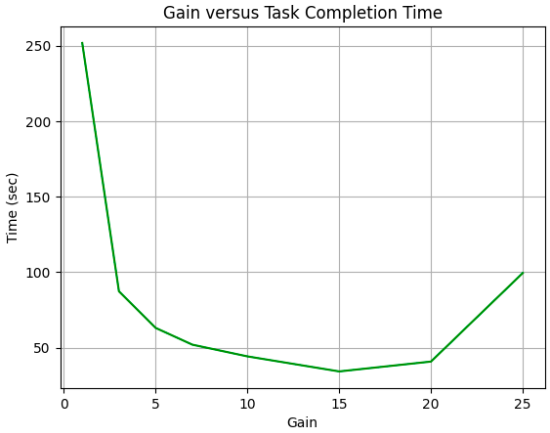


Fig. 9: Mission Duration versus Gain

actively operated to minimize the error until it reached or fell below this threshold value. By employing this approach, our system exhibited a high level of accuracy in achieving precise end-effector positioning and manipulation tasks.

5.3 Task Completion Time

To evaluate the effectiveness of our mobile manipulator system, we conducted experiments to measure the duration of task completion in different pick-and-place scenarios, considering various gain values. Figure 9 illustrates the relationship between the gain parameter and the mission duration. It was observed that increasing the gain resulted in faster task completion initially. However, beyond a certain point, further increases in gain led to an increase in the time required to accomplish the task.

6 Discussion

The results obtained from our experiments provide valuable insights into the performance and effectiveness of the mobile manipulator system for picking and placing objects. In this section, we discuss the findings and implications of our study in detail.

6.1 Effectiveness of Task Priority Algorithm

The task priority algorithm implemented in our mobile manipulator system demonstrated its effectiveness in coordinating actions and achieving accurate pick-and-place tasks. By assigning priorities to different subtasks, such as end effector position control and joint velocity control, the algorithm ensured smooth and efficient operation of the system.

However, we encountered challenges when incorporating the configuration task into the task priority algorithm. The configuration task involves not only reaching a specific position but also achieving a desired orientation for the end effector. Due to joint limits in the robot's range of motion, the task priority algorithm failed to appropriately handle this aspect. Instead of utilizing joints that were not at their limits to achieve the desired orientation, the algorithm continuously applied velocities to joints that were already at their limits. As a result, the robot was unable to achieve certain orientations specified by the configuration task.

To mitigate this issue, we focused solely on the position3D task, which involves reaching a desired position in three-dimensional space without considering the specific end effector orientation. This decision was made to ensure reliable and accurate task completion given the limitations of the task priority algorithm in handling configuration tasks.

Further research and development are needed to address the challenges encountered with the configuration task in the task priority algorithm. Alternative algorithms or control strategies could be explored to effectively incorporate both position and orientation requirements, considering the joint limits of the robot. This would enhance the capabilities of the mobile manipulator system and enable it to successfully accomplish more complex tasks involving specific end effector configurations.

6.2 Impact of Gain Parameter

The gain parameter played a critical role in determining the performance and task completion time of the mobile manipulator system. Our experimental results demonstrated that increasing the gain initially led to faster task completion. This can be attributed to the heightened responsiveness of the system to control signals.

Figure 5 illustrates the performance of the system when a gain of 1 was utilized. As depicted in Figure 5(a) and Figure 5(b), the robot executed a smooth and stable trajectory, albeit requiring 215.74 seconds to complete the task. Subsequently, Figure 6 exhibits the system's performance with a gain of 3. Despite maintaining a smooth and stable trajectory, the task completion time was reduced to 87.29 seconds.

To further examine the impact of gain, Figure 7 displays the system's behavior with a gain of 10. Noticeably, the robot's trajectory becomes slightly jerky and unstable. Nonetheless, the time required to complete the task decreased to 44.21 seconds, illustrating the influence of gain on task completion time. Finally, Figure 8 portrays the system's performance with a gain of 15. In this case, the robot's trajectory becomes significantly jerky and highly unstable Figure 8(c), yet the task completion time is further reduced to 34.25 seconds. It is worth stating, the system also started to oscillate, see Figure 8(d) and Figure 8(e).

As the gain increases, the settling time, which denotes the time required to reach a steady state with lower error values, decreases. However, an increase in gain leads to oscillations around the desired position, resulting in an increase in task completion time, as shown in Figure 9. Notably, the minimum time required was observed with a gain of 15, but beyond this point, the task completion time started to increase, indicating oscillatory behavior and a longer settling time.

In our study, we identified a gain of 5 as the optimal choice. With this gain, the robot exhibited stable motion, Figure 10, and the task completion time was reasonably low, amounting to 63.07 seconds. Therefore, careful selection and tuning of the gain parameter are crucial to ensure optimal performance of the mobile manipulator system.

6.3 Impact of Damping Factor

To address the issue of inverting the task Jacobian, we employed the Damped Least Squares (DLS) method. One

crucial parameter in this approach is the damping factor, which plays a significant role in the matrix inversion process. During our experimentation, we observed an interesting behavior concerning the damping factor.

When the damping factor is set to a small value, we encountered singularity problems during matrix inversion. This indicates that using a small damping factor results in a similar behavior to conventional matrix inversion methods, which struggle with inverting singular matrices. On the other hand, increasing the damping factor eliminates the singularity issues associated with matrix inversion.

However, we found that striking a balance with the damping factor was crucial for optimal performance. Keeping the factor too small could lead to errors in inverting the task Jacobian, potentially causing program crashes. We observed that a small damping factor, as shown in Figure 10(a), resulted in the end effector moving in a nearly straight line towards the desired position, leading to a shorter mission duration of approximately 65 seconds (Figure 10(d)).

Conversely, when the damping factor was increased to 0.5, as depicted in Figure 11(a), both the end effector and the mobile base deviated from a straight-line path and followed a curved trajectory. This significantly increased the mission duration to over 400 seconds (Figure 11(d)). Thus, while increasing the damping factor resolves the singularity problem, it also introduces unpredictability in the movement of the end effector.

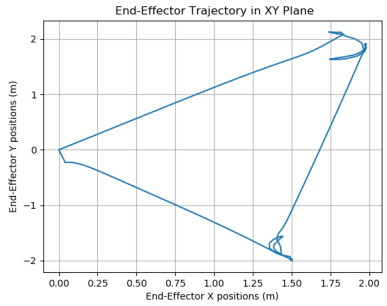
To strike an appropriate balance, we selected a damping factor of 0.05 for our project. This value allowed for successful matrix inversion while maintaining a reasonable mission duration and predictable end effector movement. It is crucial to choose an optimal damping factor to ensure efficient and accurate operation of the mobile manipulator system.

6.4 Weighted DLS

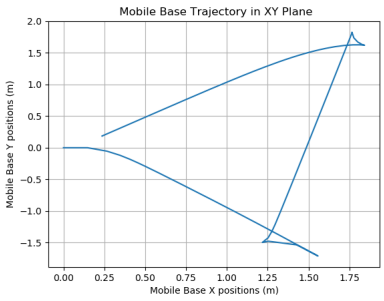
Weighted Damped Least Squares (WDLS) is a technique commonly employed in robotics to address redundancy in robot manipulators. By assigning weights to individual joints, WDLS optimizes the inverse kinematics solution, taking into account the relative importance of each joint. This approach enables flexible control over the robot's behavior while ensuring compliance with operational constraints, such as joint preference to do a task, joint limits and collision avoidance. In this project, we investigate the impact of varying these weights on the performance of the mobile manipulator system. We present three experimental scenarios to illustrate the effects.

In the first scenario (Figure 12), equal weights are assigned to both the arm and the mobile base. Consequently, the controller allocates comparable efforts to both components in accomplishing the task. However, this uniform distribution of control can lead to occasional instability. Specifically, in cases where it would be more appropriate to prioritize the movement of the base over the arm, the controller tends to utilize both components simultaneously. Similarly, when the arm should be given more preference, the controller still relies on the base. The joint velocity plot in Figure 12(e) depicts the balanced velocities sent to the arm and base.

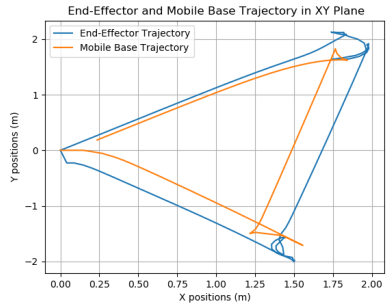
To examine the effect of altering the weights, we assign a higher weight to the arm (50) and the base (1). In this



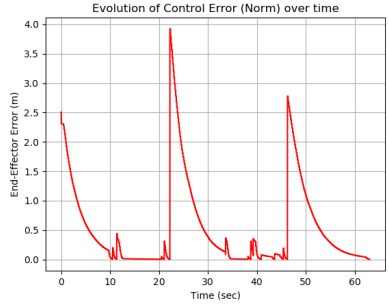
(a) End-effector Trajector in XY Plane



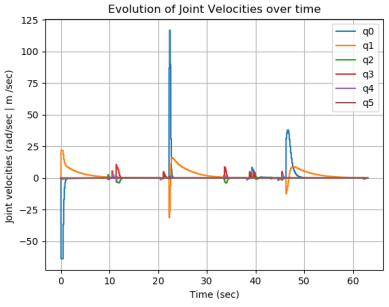
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory

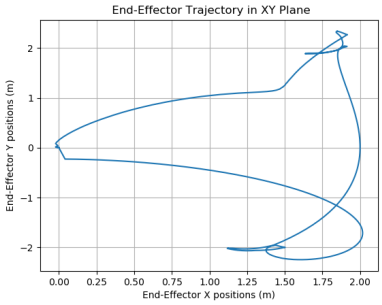


(d) End-Effector Position Error

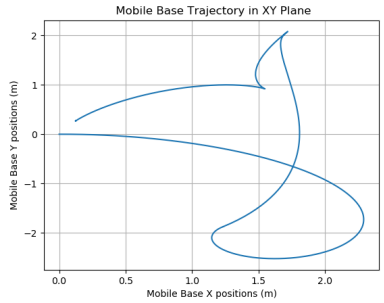


(e) Joint Velocities

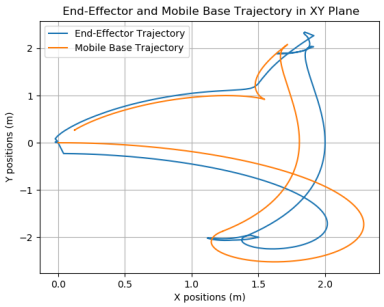
Fig. 10: Analysis: Gain 5 and Damping Factor 0.05



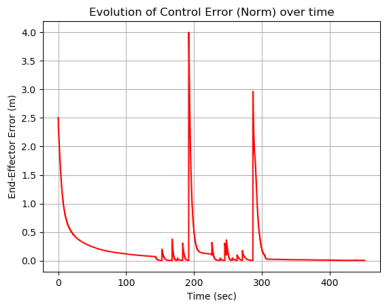
(a) End-effector Trajector in XY Plane



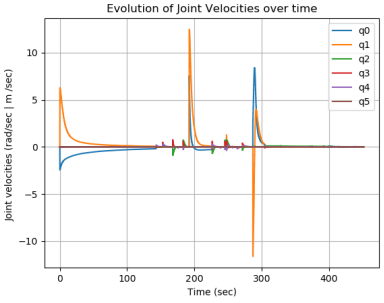
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory

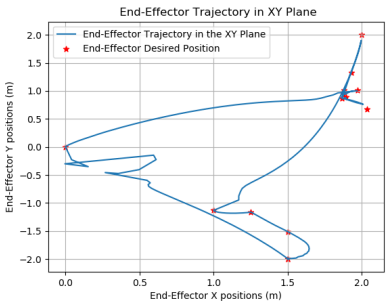


(d) End-Effector Position Error

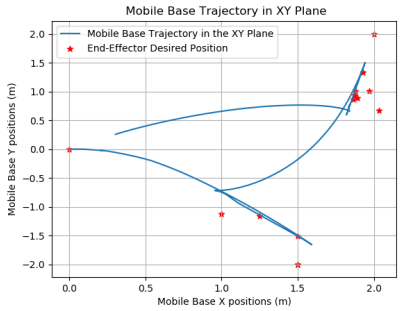


(e) Joint Velocities

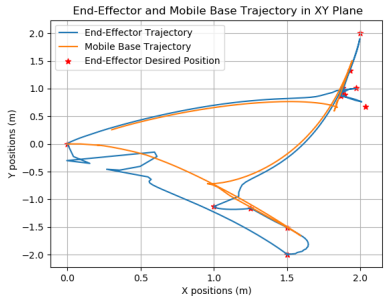
Fig. 11: Analysis: Gain 5 and Damping Factor 0.5



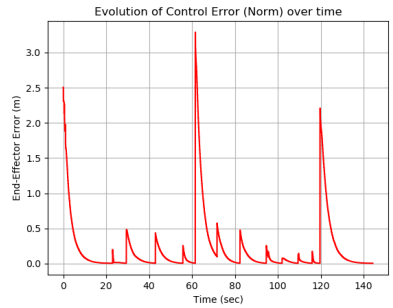
(a) End-effector Trajector in XY Plane



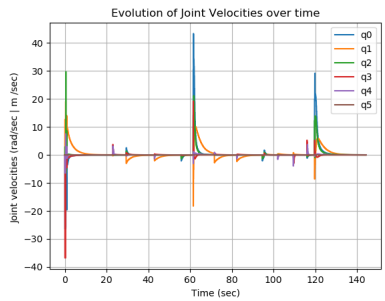
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory

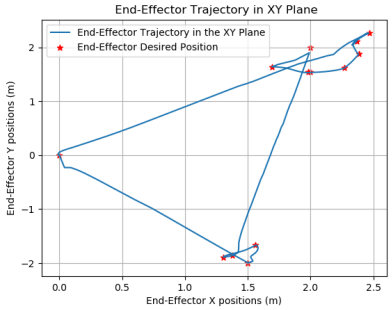


(d) End-Effector Position Error

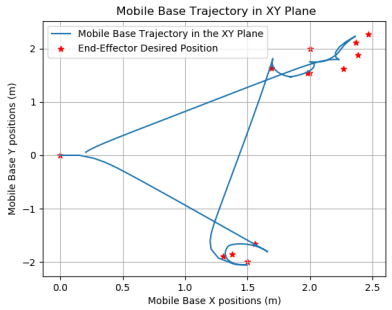


(e) Joint Velocities

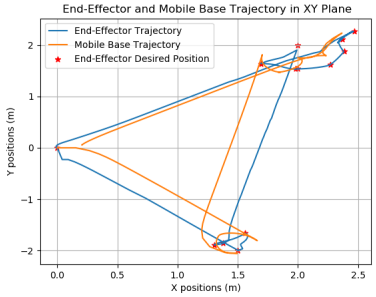
Fig. 12: Analysis: Gain 5 and Damping Factor 0.05 with Equal Weights for the Base and Arm



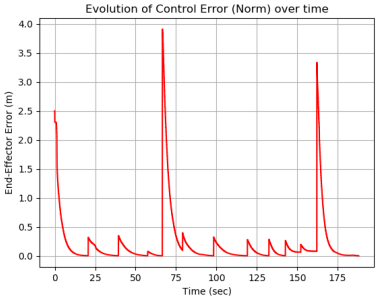
(a) End-effector Trajector in XY Plane



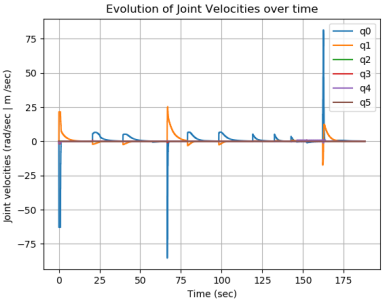
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory

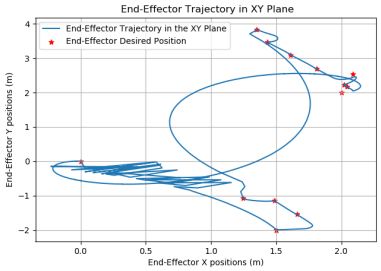


(d) End-Effector Position Error

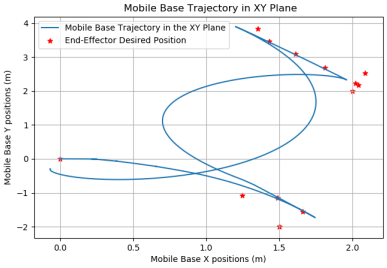


(e) Joint Velocities

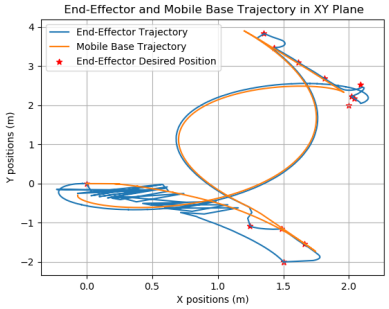
Fig. 13: Analysis: Gain 5 and Damping Factor 0.05 with Base being used more by the controller to complete the task



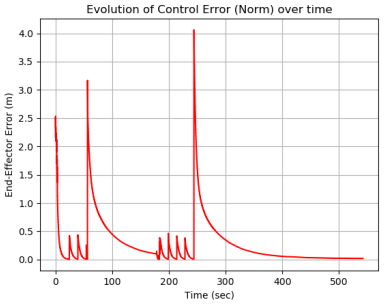
(a) End-effector Trajector in XY Plane



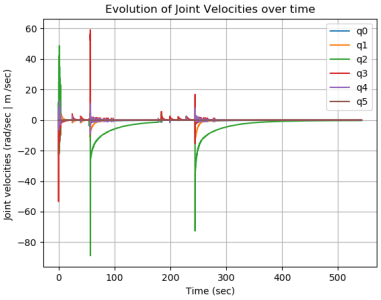
(b) Mobile base Trajector in XY Plane



(c) End-Effector and Mobile Base XY Plane Trajectory



(d) End-Effector Position Error



(e) Joint Velocities

Fig. 14: Analysis: Gain 5 and Damping Factor 0.05 with Arm being used more by the controller to complete the task

configuration (Figure 13), the controller starts to allocate more velocities to the base, as evident from the joint velocity plot in Figure 13(e). (q_0 and q_1 are base joints while the rest are the arm joints) The mobile base successfully approaches the desired positions indicated by the red asterisks in Figure 13(b). However, a challenge arises when the base reaches the desired position while the end effector struggles to achieve the same position. The base's proximity to the desired location inhibits the end effector's ability to reach it. This issue stems from the controller's inclination to prioritize the base over the arm.

Conversely, we also investigate the scenario where a higher weight is assigned to the base (10) and the arm (1). In this case (Figure 14), the controller favors the arm over the base. As a result, the system exhibits pronounced instability, characterized by significant oscillations in the motion of the end effector (Figure 14(a)). The joint velocity plot in Figure 14(e) further illustrates the controller's preference for the arm joints over the base.

To address the challenges observed in the previous scenarios, we introduce a dynamic weighting scheme. Specifically, when the distance between the end effector and the desired position exceeds 0.3m, an if condition is used to assigns a higher weight to the arm, damping its movement. As the end effector approaches the desired position, the arm weight decreases and base weights increase, enabling the arm to assume greater control. Algorithm **??** outlines the implementation of this dynamic weighting scheme, aiming to strike a balance between the use of the base and the arm based on their respective advantages at different stages of the task.

The utilization of weighted damped least squares in our mobile manipulator system presents a flexible approach to handle redundancy. By carefully tuning the weights assigned to the arm and the base, we can adapt the system's behavior to specific task requirements. The experimental results highlight the importance of striking a balance between the two components and emphasize the need for a dynamic weighting scheme to optimize performance.

6.5 Limitations

As discussed earlier, our implementation of the task priority algorithm faced limitations when attempting to achieve the end effector configuration task. The algorithm was still trying to achieve the task with joints already at their limits, making it challenging to achieve the desired configuration. In future work, it is recommended to explore more advanced algorithms specifically designed for end-effector configuration tasks.

During the simulation phase, we encountered an issue where the robot's end effector was unable to raise to the required height when grasping an object. We have provided a video demonstration that illustrates this issue, which can be accessed by clicking [HERE]. It was observed that even when the controller commanded the arm to move the end effector vertically upward, it remained almost stationary with the object in its grasp. We explored the possibility that the weight of the object might be exceeding the arm's capacity. To investigate further, we attempted to reduce the mass of the object, but encountered simulation instability issues. For instance, when setting the mass of the object to as low as 0.07, the simulation failed to function properly. Consequently, we were unable to determine if the weight of the object was indeed the cause of the end effector's inability to move upward.

Another challenge we faced was the limited time available for experimentation. Although we achieved positive results in a few trials with the real robot, we were not able to conduct extensive tests. We have recorded a video showcasing the system's performance with the real robot, which can be accessed by clicking [HERE]. Our intention was to replicate the movements of the end effector as demonstrated in the simulation, which can be viewed by clicking [HERE]. Unfortunately, time constraints prevented us from fully trying these desired movements in real robot experiments.

7 Conclusion

This paper has presented the design and implementation of a kinematic control system for a mobile manipulator, specifically a differential drive robot with a 4 DOF manipulator. Through the utilization of a Task-Priority redundancy resolution algorithm, the control system successfully addressed the challenges associated with coordinating the movements of the robot base and manipulator, particularly in the context of palletizing tasks. The integration of sensors, including wheel encoders, an RGBD camera, and a 2D lidar, along with the vacuum gripper, enhanced the capabilities of the mobile manipulator for perception and object manipulation. The project's systematic approach, comprising conceptual design, simulation-based implementation, experimental validation, and integration phases, ensured a comprehensive and reliable control system. The successful implementation and evaluation of the control system in both simulation and real-world scenarios validate its effectiveness and applicability.

Acknowledgments

The authors express their gratitude to VICOROB and UdG for their invaluable laboratory support and technical assistance regarding the utilization of devices such as cameras and robots.

References

[1] Martin Sereinig, Wolfgang Werth, and Lisa-Marie Faller. A review of the challenges in mobile manipulation: systems design and robocup challenges. *Elektrotech. Informationstechnik*, 137(6):297–308, 2020.

[2] Shantanu Thakar, Srivatsan Srinivasan, Sarah Al-Hussaini, Prahar M Bhatt, Pradeep Rajendran, Yeo Jung Yoon, Neel Dhanaraj, Rishi K Malhan, Matthias Schmid, Venkat N Krovi, et al. A survey of wheeled mobile manipulation: A decision-making perspective. *Journal of Mechanisms and Robotics*, 15(2):020801, 2023.

[3] Francisco Rubio, Francisco Valero, and Carlos Llopis-Albert. A review of mobile robots: Concepts, methods, theoretical framework, and applications. *International Journal of Advanced Robotic Systems*, 16(2):1729881419839596, 2019.

[4] Roger Bostelman, Tsai Hong, and Jeremy Marvel. Survey of research for performance measurement of mobile manipulators. *Journal of Research of the National Institute of Standards and Technology*, 121:342, 2016.

[5] Jörg Stückler, Max Schwarz, and Sven Behnke. Mobile manipulation, tool use, and intuitive interaction for cognitive service robot cosero. *Frontiers in Robotics and AI*, 3:58, 2016.

[6] Chin Pei Tang, Rajankumar Bhatt, and Venkat Krovi. Decentralized kinematic control of payload by a system of mobile manipulators. In *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004*, volume 3, pages 2462–2467. IEEE, 2004.

[7] José Varela-Aldás, Víctor Hugo Andaluz, and Fernando A Chicaiza. Modelling and control of a mobile manipulator for trajectory tracking. In *2018 International Conference on Information Systems and Computer Science (INCISCOS)*, pages 69–74. IEEE, 2018.

[8] Evangelos Papadopoulos and John Poulakakis. Planning and model-based control for mobile manipulators. In *Proceedings. 2000 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2000)(Cat. No. 00CH37113)*, volume 3, pages 1810–1815. IEEE, 2000.

[9] Heiko Engemann, Shengzhi Du, Stephan Kallweit, Patrick Cönen, and Harshal Dawar. Omnivil—an autonomous mobile manipulator for flexible production. *Sensors*, 20(24):7249, 2020.

[10] Glenn D White, Rajankumar M Bhatt, Chin Pei Tang, and Venkat N Krovi. Experimental evaluation of dynamic redundancy resolution in a nonholonomic wheeled mobile manipulator. *IEEE/ASME Transactions on Mechatronics*, 14(3):349–357, 2009.

[11] Giovanni Colucci, Andrea Botta, Luigi Tagliavini, Paride Cavallone, Lorenzo Baglieri, and Giuseppe Quaglia. Kinematic modeling and motion planning of the mobile manipulator agri. q for precision agriculture. *Machines*, 10(5):321, 2022.